

Background

Smart TVs are really annoying to work with. They're super slow, full of ads and very limited. An alternative solution where you can "self-host" a smart TV does not really exist.

Customer wants

- As a customer I would want to be able to watch anything I could watch on my smart TV.
- As a customer I want the experience similar to a dedicated smart TV, I don't want it to feel like a computer running a UI.
- As a customer I want to be able to customize the homepage without writing code.
- As a customer I want to be able to add my own apps and widgets (not necessarily without code).
- As a customer/developer of an open source repository, I would want the code not to be bloated and unmaintainable.

Problem

- How do you create a smart TV-like application, without having to create a new OS?
- Even if you were able to, how would you get apps to work?
- How do we get all the functionality, in a comfortable way?

Suggested solution

A potential solution to this issue is running custom software on a Raspberry Pi 5 running some Linux OS. With that and a custom controller or a keyboard with built in mouse, you could emulate or get close to what a smart TV does, but with way more options. The core program would include the following:

1. An "App" picker, where you can choose things such as "Plex", "Youtube" "Twitch" "Kodi" etc.
 - These apps would launch (in fullscreen) either a software (like for kodi), or the browser in kiosk mode with the corresponding app url.
2. It would also have room for "Widgets", these would be things like Plex "continue watching", the premier league table. These are really endless in what they could be, and that's part of the design.
3. **Modularity, customizability.** One of the core features would be that users through TOML or YAML can add their own widgets (they would require users adding code) and apps (should be able to be fully configured through conf file as they only need an exec command, icon and name)
4. When exiting whatever app you were using, you would go back to the UI.
5. Nice animations

Apps

The apps themselves can live as a carousel, but research needs to be done on what the most liked smart TVs use.

App ideas:

- Plex
- Youtube
- Twitch
- Kodi
- Tv4
- Svt
- Steam link
- Plexamp?

Widgets

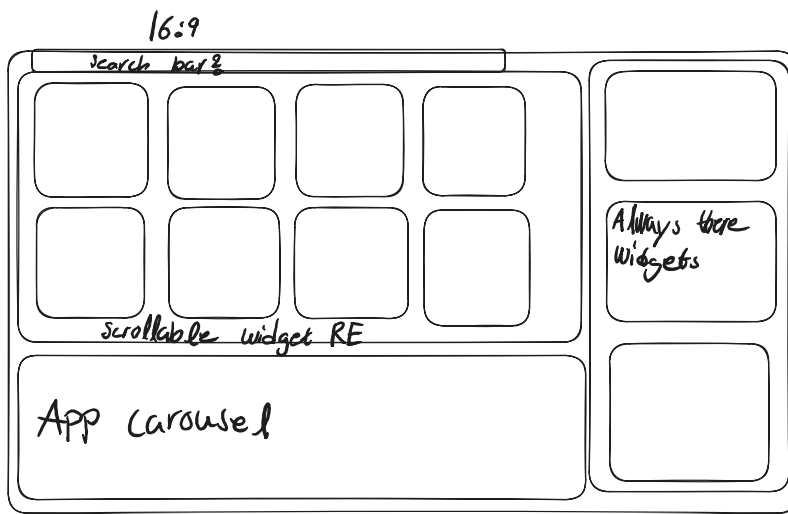
How do we give the widgets their space? How do we divide it? One idea is to have screen real estate consisting of squares that widgets can make use of. A widget can consist of any number of real estate squares, and it's up to the user how they want to use them.

Widget ideas:

- Plex continue watching
- Youtube continue watching
- Continue playing music (plexamp?)
- News
- Weather with UV index
- Premier league table
- Upcoming events in calendar
- Server status
- Upcoming movies and shows?
- When does the next bus leave outside (västtrafik)?
- Upcoming albums? Released today albums.

The home screen

How would we want to divide up the actual home screen? What do we want were? We have the concept of the carousel and the squares, however, we still have to place them. This is the sort of difficult part, and it will probably have several iterations. The first idea looks like the following image:



It consist of an app carousel at the bottom 3/4 of height and takes up about 3/4 of width. On top of that a widget carousel/scrollable thing(?) that takes up the remaining 3/4 of height. On top of that there could potentially be a search bar for any of the widgets or apps, though currently I don't see any reason for why we would want that. The remaining 1/4 of the screen could be "always there" widgets, that just always remaining visible on the home screen. This UI will very much be a work in progress with trial and error by using it and finding what is actually useful. I think having a top bar with time and some minor widgets also makes sense in my opinion. We also have to be wary of clutter.

With the real estate idea, we would need to think smartly how widgets get their real estate and how the customer places their apps and widgets where they want. I would like to avoid having an in app editor, until later on.

Application Launching

The launcher should support different action types, such as:

- Web URL
- Native command/application
- System actions

Conceptually:

```

TV Hub
↓
User selects application
↓
Configured action is executed
↓
Application takes over the screen
↓
Application exits
↓
TV Hub becomes visible again

```

The platform-specific launching behaviour should be isolated from the UI so the core application remains portable.

Movement control

The movement inside the actual app would consist of simple up, down, left or right and a select button, just as smart TVs currently work.

Tools

To build this solution, my proposal is to use the `iced` Rust GUI framework and launch it in kiosk mode when the Pi boots.

Configuration

Example TOML configuration:

```
[[apps]]
name = "Plex"
icon = "plex.png"

[apps.action]
type = "url"
url = "http://plex.local"

[[apps]]
name = "Kodi"
icon = "kodi.png"

[apps.action]
type = "command"
command = "kodi"

[[apps]]
name = "TV4"
icon = "tv4.png"
```

Configuration should eventually control:

- Applications
- Icons
- Actions
- Layout
- Number of columns
- Tile dimensions
- Gaps
- Theme

- Navigation behaviour
- Animations

Adding or rearranging applications should not require changing Rust source code.

Architecture

Keep the application separated into roughly:

```
tv-hub/  
├─ src/  
│   ├─ main.rs  
│   ├─ app.rs  
│   ├─ ui/  
│   ├─ config/  
│   ├─ apps/  
│   ├─ plex/  
│   └─ platform/  
├─ assets/  
├─ config/  
└─ Cargo.toml
```

Core application state should be independent from Pi-specific functionality.

A platform abstraction can handle:

- Application launching
- Shutdown
- Reboot
- Other OS-specific functionality

Development

Development for this project will use claude-code. However, I want to be very much a part of the development process and we therefore want to make incremental changes, verified by myself. We'll create a Github repository, where we'll make pull requests I can code review and verify before merging anything to mainline. Mainline will be protected after initial commit.

The first useful prototype should be deliberately small:

1. Iced application launches fullscreen
2. Six configurable application tiles
3. Keyboard navigation
4. Focus indicator
5. Enter/select action
6. Back/Home behaviour
7. TOML configuration

8. Basic application launching
9. Clock
10. Clean TV-oriented visual design

After that, add things such as Plex API integration, Continue Watching, recently added media, HDMI-CEC, richer animations, and additional integrations.

Developer tooling

As stated above, claude code will be used, and it will be used on WSL for the time being, unless that proves a problem.

RPi testing

For now we don't have access to a Raspberry Pi, but that will change in the near future. I want a script that does the whole pipeline of updating the code, build the arm version etc. on the RPi through ssh so that testing can be easily done.

Additional considerations

Escape hatch

Kiosk mode with no visible way out is painful to debug remotely. Build in a hidden key combo (long-press, or a specific button sequence) that drops to a settings/debug screen or a shell.

Ad-blocking for browser-launched apps

Chromium kiosk mode *can* run ad blockers, just not via the Web Store UI. Either load an unpacked extension at launch (`--load-extension=/path/to/ublock --disable-extensions-except=/path/to/ublock`) or force-install one silently via `/etc/chromium/policies/managed/policy.json` (`ExtensionInstallForcelist`). The complication is that uBlock Origin classic relies on Manifest V2, which Google is phasing out of Chrome/Chromium in favor of Manifest V3 (uBlock Origin Lite is the MV3-compatible replacement, but with fewer features — no dynamic filtering or script-blocking). Whether standalone open-source Chromium keeps MV2 support longer than Google Chrome itself is unclear and could shift under us, which is an annoyance for a "set up once" appliance.

Firefox is more attractive on ad-blocking specifically: it still fully supports MV2, so uBlock Origin classic works at full strength, kiosk mode works via `--kiosk`, and `policies.json` supports silent, non-interactive extension installs.

The catch is video decode performance on the Pi. Chromium has V4L2 hardware video decode support built in (the API most ARM SoCs, including the Pi, expose), while Firefox does not. This will have to be tested, and up to users as well.

Suggested approach: make browser choice per-app rather than global, since each app entry already has its own `action` config:

```
[apps.action]
type = "url"
url = "https://youtube.com"
browser = "chromium" # or "firefox"
```

CI for the RPi pipeline

Once the RPi deploy script (see Open questions) exists, the same cross-compile step can run in GitHub Actions on every PR, so a broken build is caught before code review rather than after.

Things to prototype first

Before committing much of the architecture to code, these are worth spiking in isolation:

1. **Compositor / window-switching model.** This is the biggest unknown and shapes almost everything else. iced needs X11, Wayland, or a raw framebuffer to render into. To have the hub disappear and a browser or Kodi take over fullscreen, then return cleanly, decide between: a minimal Wayland compositor (e.g. `cage` or `labwc`) hosting both the hub and launched apps, switching focus/visibility between them — or VT switching / process kill-and-restart, with only one process ever owning the display. Prototype this before writing much of the hub UI itself.
2. **Launch → return latency and cleanliness.**
3. **HDMI-CEC in practice.**
4. **Real remote/controller input, not just keyboard.**
5. **Animations.** Check the current state of iced's animation API before committing to "nice animations" as a v1 goal.

TODO

- ✓ Test out iced ✓ 2026-08-29
- ✓ Test continue watching plex api ✓ 2026-08-29
 - This proved a bit problematic, as we have to pick a user, and it doesn't actually start whatever we want to continue watching. Have a potential workaround where we
 1. Start a browser with Plex
 2. The program polls waiting for the user to pick user etc.
 3. Program uses the plex companion API to start the viewing
- ✓ Test premier league table ✓ 2026-08-29
- ✓ Test keyboard navigation ✓ 2026-08-29

Open questions

- What OS do we run on the Pi?
 - **Suggestion:** Raspberry Pi OS Lite (64-bit, Bookworm). It has the best-supported GPU drivers for Pi 5 (important for iced's rendering backend and for Chromium kiosk mode), the most documentation, and won't fight hardware quirks. A minimal custom image (Buildroot/Yocto) would boot faster and be "purer," but for an open source hobby project that others will want to replicate, the maintenance burden isn't worth it yet — revisit later if boot time becomes a real complaint. Disable unnecessary services (Bluetooth/avahi if unused) and autostart the binary via a systemd unit rather than a full desktop session.
- What should the Pi specs be?
 - Raspberry Pi 5 8GB Ram with SSD add-on and cooler is my bad idea
 - **Suggestion:** 8GB is more than the core UI needs, but Chromium in kiosk mode (for YouTube/Twitch/web apps) is the actual RAM hog, so 8GB is a reasonable buffer rather than overkill. NVMe/SSD boot mainly helps reduce SD card wear on an always-on device, more than it helps speed. Active cooling makes sense given this runs 24/7.
- How do we distinguish between apps and widgets?
 - **Suggestion:** codify it as a lifecycle difference rather than a content difference: an *app* takes exclusive input focus and the hub becomes invisible until it exits; a *widget* never takes input focus (or only very limited, contextual input) and lives inside the hub's layout with its own refresh/render cycle. This gives a clean trait boundary in Rust — `LaunchableApp` vs `Widget`.
- How do we decide on screen real estate? TV's are huge, so we should have quite a lot to work on.
 - **Suggestion:** a bento/grid system. Define a base grid (e.g. 12 columns × 8 rows), let each widget declare a min/max size in grid units, and write a simple packing algorithm (a naive greedy top-left-fill is fine for v1). Home Assistant's Lovelace dashboard and MagicMirror's module system already solve a similar problem and are worth skimming for prior art before designing this from scratch.
- TOML or YAML?
 - **Suggestion:** TOML. Better Rust ecosystem support (the `toml` crate is mature and integrates cleanly with `serde`), no YAML footguns (implicit type coercion, the "Norway problem," indentation sensitivity), and the example config in this doc already reads naturally in TOML.

Additional open question: Plex continue-watching approach

The polling + manual user-pick workaround (noted in the TODO) is reasonable but a bit fragile. Before locking it into the architecture, it's worth checking how other Plex client projects handle auth — storing a long-lived `X-Plex-Token` for the hub itself (rather than per-browser-session) may allow calling the local Plex server API directly for continue-watching data and playback initiation, without the companion-API dance.

Additional open question: RPi deploy pipeline

For the "test on RPi via SSH" tooling: cross-compile for `aarch64-unknown-linux-gnu` (via `cross` or `cargo-zigbuild` so a physical Pi isn't required to build), `scp` the binary over, and restart the systemd service via `ssh`. A `deploy.sh` script covering this could double as the basis for a GitHub Actions cross-compile check on every PR.